

part C. Unions and Literals

Earlier we covered the concept of the “type system” and how it can read values to understand the types of variables. Now I’d like to introduce two key concepts that TypeScript works with to make inferences on top of those values:

Unions

Expanding a value’s allowed type to be two or more possible types

Narrowing

Reducing a value’s allowed type to *not* be one or more possible types

Put together, unions and narrowing are powerful concepts that allow TypeScript to make informed inferences on your code many other mainstream languages cannot.

Union Types

Take this `mathematician` variable:

```
let mathematician = Math.random() > 0.5
  ? undefined
  : "Mark Goldberg";
```

What type is `mathematician`?

It’s neither only `undefined` nor only `string`, even though those are both potential types. `mathematician` can be *either* `undefined` or `string`. This kind of “either or” type is called a *union*. Union types are a wonderful concept that let us handle code cases where we don’t know exactly which type a value is, but do know it’s one of two or more options.

TypeScript represents union types using the `|` (pipe) operator between the possible values, or *constituents*. The previous `mathematician` type is thought of as `string | undefined`. Hovering over the `mathematician` variable would show its type as `string | undefined`

```
let mathematician: string | undefined

let mathematician = Math.random() > 0.5
  ? undefined
  : "Mark Goldberg";
```

Figure 3-1. TypeScript reporting the `mathematician` variable as being type `string | undefined`

Declaring Union Types

Union types are an example of a situation when it might be useful to give an explicit type annotation for a variable even though it has an initial value. In this example, `thinker` starts off `null` but is known to potentially contain a `string` instead. Giving it an explicit `string | null` type annotation means TypeScript will allow it to be assigned values of type `string`:

```
let thinker: string | null = null;

if (Math.random() > 0.5) {
  thinker = "Susanne Langer"; // Ok
}
```

Union type declarations can be placed anywhere you might declare a type with a type annotation.

Union Properties

When a value is known to be a union type, TypeScript will only allow you to access member properties that exist on all possible types in the union. It will give you a type-checking error if you try to access a member property that doesn't exist on all possible types.

In the following snippet, `physicist` is of type `number | string`. While `.toString()` exists in both types and is allowed to be used, `.toUpperCase()` and `.toFixed()` are not because `.toUpperCase()` is missing on the `number` type and `.toFixed()` is missing on the `string` type:

```
let physicist = Math.random() > 0.5
  ? "Marie Curie"
  : 84;

physicist.toString(); // Ok
```

```
physicist.toUpperCase();  
//      ~~~~~  
// Error: Property 'toUpperCase' does not exist on type 'string | number'.  
//   Property 'toUpperCase' does not exist on type 'number'.  
  
physicist.toFixed();  
//      ~~~~~  
// Error: Property 'toFixed' does not exist on type 'string | number'.  
//   Property 'toFixed' does not exist on type 'string'.
```

Blocking access to properties that don't exist on all union types is a safety measure. If an object is not known to definitely be a type that contains a property, TypeScript will believe it unsafe to try to use that property. The property might not exist!

To use a property of a union typed value that only exists on a subset of the potential types, your code will need to indicate to TypeScript that the value at that location in code is one of those more specific types: a process called *narrowing*.

Narrowing

Narrowing is when TypeScript infers from your code that a value is of a more specific type than what it was defined, declared, or previously inferred as. Once TypeScript knows that a value's type is more narrow than previously known, it will allow you to treat the value like that more specific type. A logical check that can be used to narrow types is called a *type guard*.

Let's cover two of the common type guards TypeScript can use to deduce type narrowing from your code.

Assignment Narrowing

If you directly assign a value to a variable, TypeScript will narrow the variable's type to that value's type.

Here, the `admiral` variable is declared initially as a `number | string`, but after being assigned the value `"Grace Hopper"`, TypeScript knows it must be a `string`:

```
let admiral: number | string;
```

```

admiral = "Grace Hopper";

admiral.toUpperCase(); // Ok: string

admiral.toFixed();
//      ~~~~~~
// Error: Property 'toFixed' does not exist on type 'string'.

```

Assignment narrowing comes into play when a variable is given an explicit union type annotation and an initial value too. TypeScript will understand that while the variable may later receive a value of any of the union typed values, it starts off as only the type of its initial value.

In the following snippet, `inventor` is declared as type `number | string`, but TypeScript knows it's immediately narrowed to a `string` from its initial value:

```

let inventor: number | string = "Hedy Lamarr";

inventor.toUpperCase(); // Ok: string

inventor.toFixed();
//      ~~~~~~
// Error: Property 'toFixed' does not exist on type 'string'.

```

Conditional Checks

A common way to get TypeScript to narrow a variable's value is to write an `if` statement checking the variable for being equal to a known value. TypeScript is smart enough to understand that inside the body of that `if` statement, the variable must be the same type as the known value:

```

// Type of scientist: number | string
let scientist = Math.random() > 0.5
  ? "Rosalind Franklin"
  : 51;

if (scientist === "Rosalind Franklin") {
  // Type of scientist: string
  scientist.toUpperCase(); // Ok
}

// Type of scientist: number | string
scientist.toUpperCase();

```

```
// ~~~~~~  
// Error: Property 'toUpperCase' does not exist on type 'string | number'.  
//   Property 'toUpperCase' does not exist on type 'number'.
```

Narrowing with conditional logic shows TypeScript's type-checking logic mirroring good JavaScript coding patterns. If a variable might be one of several types, you'll generally want to check its type for being what you need. TypeScript is forcing us to play it safe with our code. Thanks, TypeScript!

Typeof Checks

In addition to direct value checking, TypeScript also recognizes the `typeof` operator in narrowing down variable types.

Similar to the `scientist` example, checking if `typeof researcher` is `"string"` indicates to TypeScript that the type of `researcher` must be `string`:

```
let researcher = Math.random() > 0.5  
  ? "Rosalind Franklin"  
  : 51;  
  
if (typeof researcher === "string") {  
  researcher.toUpperCase(); // Ok: string  
}
```

Logical negations from `!` and `else` statements work as well:

```
if (!(typeof researcher === "string")) {  
  researcher.toFixed(); // Ok: number  
} else {  
  researcher.toUpperCase(); // Ok: string  
}
```

Those code snippets can be rewritten with a ternary statement, which is also supported for type narrowing:

```
typeof researcher === "string"  
  ? researcher.toUpperCase() // Ok: string  
  : researcher.toFixed(); // Ok: number
```

Whichever way you write them, `typeof` checks are a practical and often used way to narrow types.

TypeScript's type checker recognizes several more forms of narrowing that we'll see in later parts.

Literal Types

Now that I've shown union types and narrowing for working with values that may be two or more potential types, I'd like to go in the opposite direction by introducing *literal types*: more specific versions of primitive types.

Take this `philosopher` variable:

```
const philosopher = "Hypatia";
```

What type is `philosopher`?

At first glance, you might say `string`—and you'd be correct. `philosopher` is indeed a `string`.

But! `philosopher` is not just any old `string`. It's specifically the value `"Hypatia"`. Therefore, the `philosopher` variable's type is technically the more specific `"Hypatia"`.

Such is the concept of a *literal type*: the type of a value that is known to be a specific value of a primitive, rather than any of those primitive's values at all. The primitive type `string` represents the set of all possible strings that could ever exist; the literal type `"Hypatia"` represents just that one string.

If you declare a variable as `const` and directly give it a literal value, TypeScript will infer the variable to be that literal value as a type. This is why, when you hover a mouse over a `const` variable with an initial literal value in an IDE such as VS Code, it will show you the variable's type as that literal

```
const mathematician: "Mark Goldberg"
```

```
const mathematician = "Mark Goldberg";
```

Figure 3-2. TypeScript reporting a `const` variable as being specifically its literal type


```
let mathematician: string  
let mathematician = "Mark Goldberg";
```

Figure 3-3. TypeScript reporting a `let` variable as being generally its primitive type

You can think of each *primitive* type as a *union* of every possible matching *literal* value. In other words, a primitive type is the set of all possible literal values of that type.

Other than the `boolean`, `null`, and `undefined` types, all other primitives such as `number` and `string` have an infinite number of literal types. The common types you'll find in typical TypeScript code are just those:

- `boolean`: just `true` | `false`
- `null` and `undefined`: both just have one literal value, themselves
- `number`: `0` | `1` | `2` | ... | `0.1` | `0.2` | ...
- `string`: `"` | `"a"` | `"b"` | `"c"` | ... | `"aa"` | `"ab"` | `"ac"` | ...

Union type annotations can mix and match between literals and primitives. A representation of a lifespan, for example, might be represented by any `number` or one of a couple known edge cases:

```
let lifespan: number | "ongoing" | "uncertain";  
  
lifespan = 89; // Ok  
lifespan = "ongoing"; // Ok  
  
lifespan = true;  
// Error: Type 'true' is not assignable to  
// type 'number | "ongoing" | "uncertain"'
```

Literal Assignability

You've seen how different primitive types such as `number` and `string` are not assignable to each other. Similarly, different literal types within the same primitive type—e.g., `0` and `1`—are not assignable to each other.

In this example, `specificallyAda` is declared as being of the literal type `"Ada"`, so while the value `"Ada"` may be

given to it, the types `"Byron"` and `string` are not assignable to it:

```
let specificallyAda: "Ada";

specificallyAda = "Ada"; // Ok

specificallyAda = "Byron";
// Error: Type '"Byron"' is not assignable to type '"Ada"'.

let someString = ""; // Type: string

specificallyAda = someString;
// Error: Type 'string' is not assignable to type '"Ada"'.
```

Literal types are, however, allowed to be assigned to their corresponding primitive types. Any specific literal string is still a `string`.

In this code example, the value `":)"`, which is of type `":)"`, is being assigned to the `someString` variable previously inferred to be of type `string`:

```
someString = ":)";
```

Who would have thought a simple variable assignment would be so theoretically intense?

Strict Null Checking

The power of narrowed unions with literals is particularly visible when working with potentially undefined values, an area of type systems TypeScript refers to as *strict null checking*. TypeScript is part of a surge of modern programming languages that utilize strict null checking to fix the dreaded “billion-dollar mistake.”

The “billion-dollar mistake” is a catchy industry term for many type systems allowing null values to be used in places that require a different type. In languages without strict null checking, code like this example that assign `null` to a `string` is allowed:

```
const firstName: string = null;
```


If you've previously worked in a typed language such as C++ or Java that suffers from the billion-dollar mistake, it may be surprising to you that some languages don't allow such a thing. If you've never worked in a language without strict null checking before, it may be surprising that some languages allowed the billion-dollar mistake in the first place!

The TypeScript compiler contains a multitude of options that allow changing how it runs. will cover TypeScript compiler options in depth. One of the most useful opt-in options, `strictNullChecks`, toggles whether strict null checking is enabled. Roughly speaking, disabling `strictNullChecks` adds `| null | undefined` to every type in your code, thereby allowing any variable to receive `null` or `undefined`.

With the `strictNullChecks` option set to `false`, the following code is considered totally type safe. That's wrong, though; `nameMaybe` might be `undefined` when `.toLowerCase` is accessed from it:

```
let nameMaybe = Math.random() > 0.5
  ? "Tony Hoare"
  : undefined;

nameMaybe.toLowerCase();
// Potential runtime error: Cannot read property 'toLowerCase' of undefined.
```

With strict null checking enabled, TypeScript sees the potential crash in the code snippet:

```
let nameMaybe = Math.random() > 0.5
  ? "Tony Hoare"
  : undefined;

nameMaybe.toLowerCase();
// Error: Object is possibly 'undefined'.
```

Without strict null checking enabled, it's much harder to know whether your code is safe from errors due to accidentally `null` or `undefined` values.

TypeScript best practice is generally to enable strict null checking. Doing so helps prevent crashes and eliminates the billion-dollar mistake.

Truthiness Narrowing

Recall from JavaScript that *truthiness*, or being *truthy*, is whether a value would be considered `true` when evaluated in a `Boolean` context, such as an `&&` operator or `if` statement. All values in JavaScript are truthy except for those defined as *falsy*: `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, and `NaN`.

TypeScript can also narrow a variable's type from a truthiness check if only some of its potential values may be truthy. In the following snippet, `geneticist` is of type `string | undefined`, and because `undefined` is always falsy, TypeScript can deduce that it must be of type `string` within the `if` statement's body:

```
let geneticist = Math.random() > 0.5
  ? "Barbara McClintock"
  : undefined;

if (geneticist) {
  geneticist.toUpperCase(); // Ok: string
}

geneticist.toUpperCase();
// Error: Object is possibly 'undefined'.
```

Logical operators that perform truthiness checking work as well, namely `&&` and `?.`:

```
geneticist && geneticist.toUpperCase(); // Ok: string | undefined
geneticist?.toUpperCase(); // Ok: string | undefined
```

Unfortunately, truthiness checking doesn't go the other way. If all we know about a `string | undefined` value is that it's falsy, that doesn't tell us whether it's an empty string or `undefined`.

Here, `biologist` is of type `false | string`, and while it can be narrowed down to just `string` in the `if` statement body, the `else` statement body knows it can still be a string if it's `""`:

```
let biologist = Math.random() > 0.5 && "Rachel Carson";
```

```
if (biologist) {  
    biologist; // Type: string  
} else {  
    biologist; // Type: false | string  
}
```

Variables Without Initial Values

Variables declared without an initial value default to `undefined` in JavaScript. That presents an edge case in the type system: what if you declare a variable to be a type that doesn't include `undefined`, then try to use it before assigning a value?

TypeScript is smart enough to understand that the variable is `undefined` until a value is assigned. It will report a specialized error message if you try to use that variable, such as by accessing one of its properties, before assigning a value:

```
let mathematician: string;  
  
mathematician?.length;  
// Error: Variable 'mathematician' is used before being assigned.  
  
mathematician = "Mark Goldberg";  
mathematician.length; // Ok
```

Note that this reporting doesn't apply if the variable's type includes `undefined`. Adding `| undefined` to a variable's type indicates to TypeScript that it doesn't need to be defined before use, as `undefined` is a valid type for its value.

The previous code snippet wouldn't emit any errors if the type of `mathematician` is `string | undefined`:

```
let mathematician: string | undefined;  
  
mathematician?.length; // Ok  
  
mathematician = "Mark Goldberg";  
mathematician.length; // Ok
```

Type Aliases

Most union types you'll see in code will generally only have two or three constituents. However, you may sometimes find a use for longer union types that are inconvenient to type out repeatedly.

Each of these variables can be one of five possible types:

```
let rawDataFirst: boolean | number | string | null | undefined;  
let rawDataSecond: boolean | number | string | null | undefined;  
let rawDataThird: boolean | number | string | null | undefined;
```

TypeScript includes *type aliases* for assigning easier names to reused types. A type alias starts with the `type` keyword, a new name, `=`, and then any type. By convention, type aliases are given names in PascalCase:

```
type MyName = ...;
```

Type aliases act as a copy-and-paste in the type system. When TypeScript sees a type alias, it acts as if you'd typed out the actual type the alias was referring to. The previous variables' type annotations could be rewritten to use a type alias for the long union type:

```
type RawData = boolean | number | string | null | undefined;  
  
let rawDataFirst: RawData;  
let rawDataSecond: RawData;  
let rawDataThird: RawData;
```

That's a lot easier to read!

Type aliases are a handy feature to use in TypeScript whenever your types start getting complex. For now, that just includes long union types; later on it will include array, function, and object types.

Type Aliases Are Not JavaScript

Type aliases, like type annotations, are not compiled to the output JavaScript. They exist purely in the TypeScript type system.

The previous code snippet would compile to roughly this JavaScript:

```
let rawDataFirst;  
let rawDataSecond;  
let rawDataThird;
```

Because type aliases are purely in the type system, you cannot reference them in runtime code. TypeScript will let you know with a type error if you are trying to access something that won't exist at runtime:

```
type SomeType = string | undefined;  
  
console.log(SomeType);  
//           ~~~~~  
// Error: 'SomeType' only refers to a type, but is being used as a value here.
```

Type aliases exist purely as a development-time construct.

Combining Type Aliases

Type aliases may reference other type aliases. It can sometimes be useful to have type aliases refer to each other, such as when one type alias is a union of types that includes (is a superset of) the union types within another type alias.

This `IdMaybe` type is a union of the types within `Id` as well as `undefined` and `null`:

```
type Id = number | string;  
  
// Equivalent to: number | string | undefined | null  
type IdMaybe = Id | undefined | null;
```

Type aliases don't have to be declared in order of usage. You can have a type alias declared earlier in a file reference an alias declared later in the file.

The previous code snippet could be rewritten to have `IdMaybe` come before `Id`:

```
type IdMaybe = Id | undefined | null; // Ok  
type Id = number | string;
```

In this part, you went over union and literal types in TypeScript, along with how its type system can deduce more specific (narrower) types from how our code is structured:

- How union types represent values that could be one of two or more types
- Explicitly indicating union types with type annotations
- How type narrowing reduces the possible types of a value
- The difference between `const` variables with literal types and `let` variables with primitive types
- The “billion-dollar mistake” and how TypeScript handles strict null checking
- Using explicit `| undefined` to represent values that might not exist
- Implicit `| undefined` for unassigned variables
- Using type aliases to save typing long type unions repeatedly